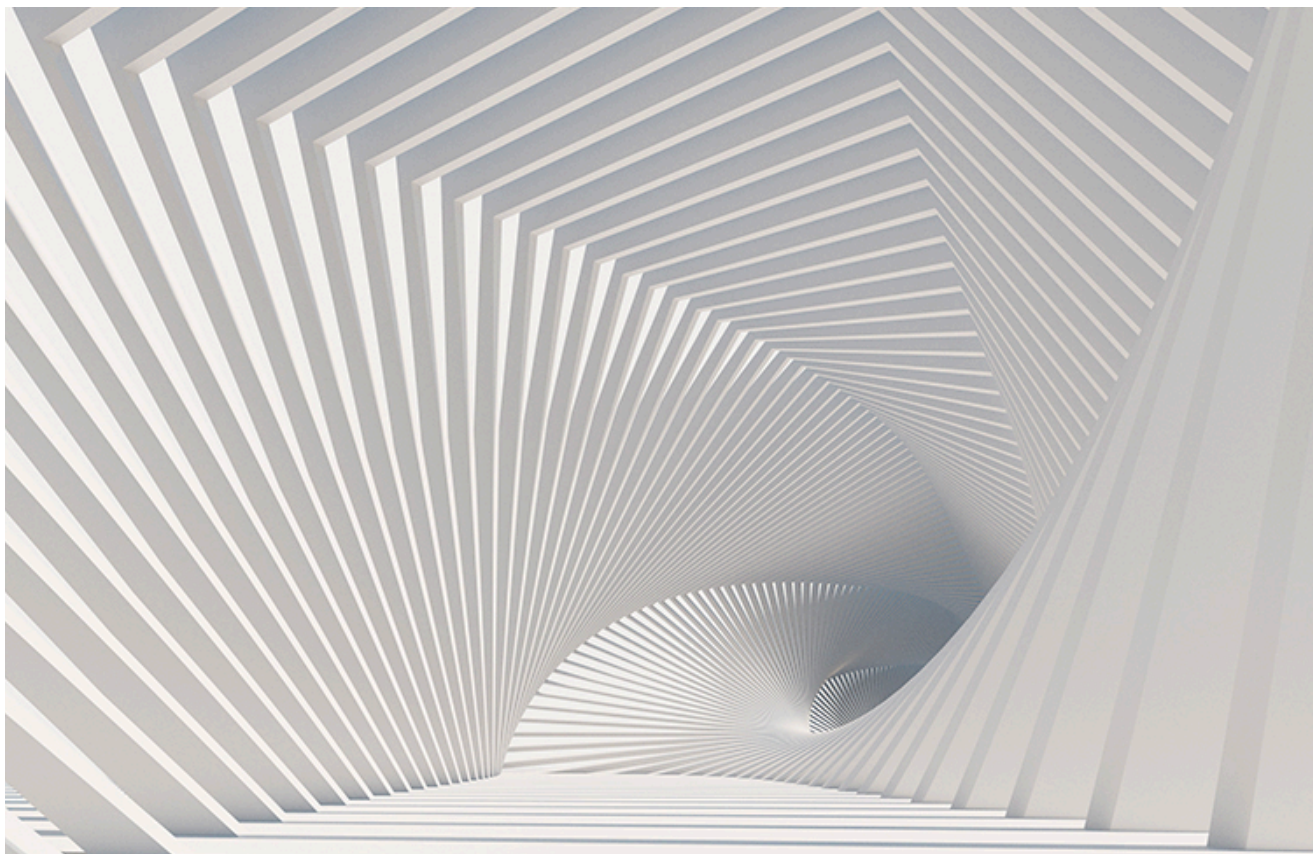


NADIA



Description

Ce document va servir de cadre général aux différentes phases de développement de l'application Nadia.

Nadia sera une assistante virtuelle alimentée par une IA locale (type LLaMA) pour aider à gérer les emails de manière sécurisée et productive. Elle pourra connecter des boîtes mail (Gmail d'abord), résumer les messages, proposer des réponses, classer/archiver les emails et identifier les plus importants, le tout sans envoyer de données à l'extérieur (l'IA fonctionne en local) . Ce document détaille un workflow découpé en phases (macro-tâches), chacune déclinée en issues GitLab pour guider un développeur.

Etude initiale et choix technologique

On va d'abord développer la partie back, l'intelligence et les apis de base. Puis le front ensuite une fois le backend opérationnel.

Contraintes :

- IA locale
- Connexion GMail d'abord
- Desktop, puis intégration web plus tard
- Stack pas trop exotique
- Tout en local (securisation)

Backend Python (IA + Gmail)

- Librairie de binding avec llama, qui semble assez riche : <https://github.com/abetlen/llama-cpp-python>
- Gmail: google fournit un QuickStart adapté pour python et la gmail api: <https://developers.google.com/workspace/gmail/api/quickstart/python>
- Backend a exposer en http pour être consommé par une application front aussi en local , ou une extension web , via FastAPI <https://fastapi.tiangolo.com/>
 - Consommation par une app Tauri ou Electron
 - Extension ou spa plus tard

Frontend Electron

- UI full node js, html css (voir si besoin d'un framework ou pas)
- Paqueter l'ui sur du electron,
- Générer les .exe et .dmg pour windows et macOS : utiliser electron-builder
- L'extension web sera vu plus tard

Empaquetage .exe et .dmg

[Electron App]

- ↳ lance le backend Python (FastAPI ou script Python)
- ↳ communique en local (localhost:port ou IPC)

[Backend Python]

↳ contient l'IA locale (llama-cpp-python + modèles)

.exe et .dmg

- electron-builder pour générer le .exe
 - Python a package via PyInstaller ou Nuitka
 - Electron embarque ce package dans le package final (via un script)
 - Le modèle llama est embarqué dans un dossier ressources/
 - L'exécutable electron démarre le back python
- > Tout doit donc tourner localement, le client a pas besoin de python sur sa machine ou autre installation externe.**

Particularité du modèle lourd

- Modèle la peut être lourd : .gguf peut être lourd (2-8Go)
 - Téléchargement différentiel, via launcher , après installation

Particularité macOS

- .dmg destiné à des gens
 - Il faut signer + notariées auprès apple

Particularité Windows

- Antivirus parfois chiant avec PyInstaller, voir si Nuitka permet de produire un binaire C++ plus propre

Proposition du modèle d'IA à implémenter

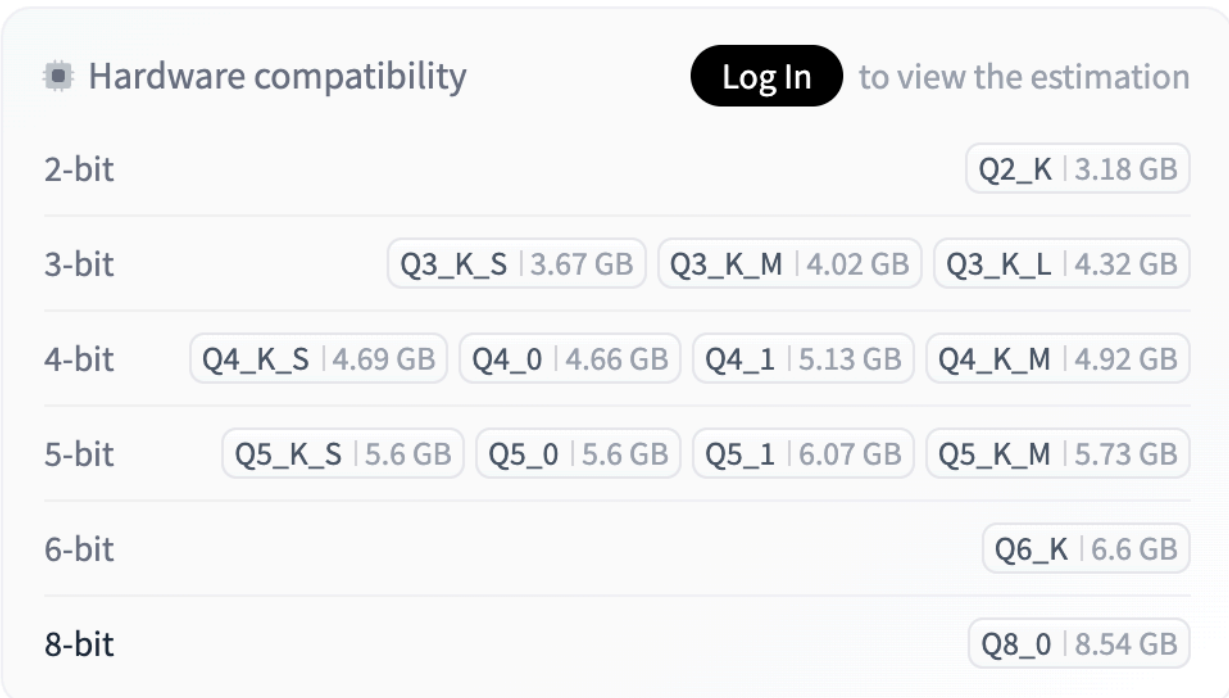
Nadia a besoin d'un modèle d'IA qui doit :

- être gratuit
- Transparent (donc open source)
- Tourner sur une machine de base (pc de mr tout le monde)
- Comprendre un contexte conversationnel grand comme des mails

- Proposer, résumer et identifier des mails (prompt following léger mais pertinent, on va pas papoter avec)

Llama 3 8B

- Cerveau téléchargeable en GGUF directement prêt à être exploité par le back python : <https://huggingface.co/QuantFactory/Meta-Llama-3-8B-GGUF> Parmi les différentes quantization, je pense qu'on peut partir sur la Q4_K_M car bon compromis taille/perf pour tourner sur l'ordi de monsieur tout le monde. (Il ne faut pas que l'outil consomme trop de RAM ! On pourra revoir cela à l'usage)



Hardware compatibility		Log In	to view the estimation
2-bit	Q2_K 3.18 GB		
3-bit	Q3_K_S 3.67 GB Q3_K_M 4.02 GB Q3_K_L 4.32 GB		
4-bit	Q4_K_S 4.69 GB Q4_0 4.66 GB Q4_1 5.13 GB Q4_K_M 4.92 GB		
5-bit	Q5_K_S 5.6 GB Q5_0 5.6 GB Q5_1 6.07 GB Q5_K_M 5.73 GB		
6-bit	Q6_K 6.6 GB		
8-bit	Q8_0 8.54 GB		

- Performant en local et sans besoin de GPU, motorisé via llama.cpp <https://github.com/ggml-org/llama.cpp>
- Interfacer avec python via llama-cpp-python <https://github.com/abetlen/llama-cpp-python>

Proposition de l'architecture globale

Architecture globale hexagonale, a représenter dans la structure du code source, voir diagramme mermaid : <https://gitlab.com/plrconseil-apps/nadIA/-/wikis/Architecture>

Proposition de l'architecture projet

Pour illustrer un commit a été réalisé pour proposer a quoi pourrait ressembler l'architecture du projet . A implementer

nadia/

```

├── backend/  # Backend Python (FastAPI + Core IA + Gmail + LLaMA)
|   ├── core/      # Domaine métier (résumé, importance, réponses)
|   |   ├── models/
|   |   |   ├── email.py
|   |   |   ├── summary.py
|   |   |   ├── importance.py
|   |   |   └── classification.py
|   |   └── services/
|   |       ├── summarization_service.py
|   |       ├── reply_service.py
|   |       ├── importance_service.py
|   |       └── classification_service.py
|   |   └── mailbox_service.py
|   |   └── __init__.py
|   |
|   ├── ports/      # Interfaces (Dépendances externes abstraites)
|   |   ├── email_provider.py
|   |   ├── llm_client.py
|   |   └── __init__.py
|   |
|   └── adapters/    # Implémentations concrètes (Gmail, LLM, Storage)
|       ├── gmail_adapter.py

```

```

| | |—— llama_cpp_adapter.py
| | |—— sqlite_storage.py
| |—— __init__.py
| |
| |—— api/          # API FastAPI exposée au frontend desktop
| | |—— main.py    # Boot FastAPI
| | |—— routers/
| | | |—— emails_router.py
| | | |—— auth_router.py
| | | |—— llm_router.py
| | |—— schemas.py # Pydantic DTO (definition ce qui rentre et sort de fastapi)
| | |—— __init__.py
| |
| |—— config/
| | |—— settings.py # chemins modèles, tokens, options LLM etc.
| | |—— logging.conf # gestion des logs (debug, error, etc)
| | |
| |—— utils/
| | |—— text_cleaner.py # nettoyage HTML → texte
| | |—— email_parser.py
| | |—— ...
| | |—— __init__.py
| |
| |—— __init__.py
| |
| |—— ressources/      # Modèles LLaMA (GGUF) .gitignore ?
| | |—— llama3-8b.Q4_K_M.gguf
| |—— tests/          # Tests unitaires backend (pytest)
| | |—— test_services.py
| | |—— test_adapters.py
| | |—— test_api.py
| |
| |—— requirements.txt

```

```

|   ├── pyproject.toml
|   └── README.md
|
|── scripts/          # Scripts utiles (dev, quantization, tests)
|   ├── download_model.sh
|   ├── benchmark_llm.py
|   ├── build_backend.sh
|   ├── build_desktop.sh
|   └── build_all.sh
|       └── ...
|
|── desktop/          # Desktop App (Electron ou Tauri)
|   ├── src/
|   │   ├── main/      # Process principal
|   │   │   ├── index.ts  # Lance le backend Python packagé
|   │   │   ├── backend_launcher.ts
|   │   │   └── menu.ts
|   │   ├── renderer/   # UI Frontend (React, Svelte, Vue...)
|   │   │   ├── pages/
|   │   │   │   ├── Inbox.tsx
|   │   │   │   ├── EmailDetail.tsx
|   │   │   │   └── Settings.tsx
|   │   │   ├── components/
|   │   │   │   ├── EmailCard.tsx
|   │   │   │   ├── SummaryBox.tsx
|   │   │   │   ├── ReplyBox.tsx
|   │   │   │   └── Loader.tsx
|   │   │   └── styles/
|   │   └── assets/
|   │
|   └── └── README.md
|
|── packaging/        # Scripts pour créer .exe et .dmg

```

```

|   |—— build_backend.spec   # PyInstaller spec
|   |—— build_backend.sh
|   |—— build_desktop.sh
|   |—— build_all.sh
|
|—— ci/                # CI/CD (GitLab CI)
|   |—— gitlab-ci.yml
|   |—— pipelines/
|       |—— test_backend.yml
|       |—— build_desktop.yml
|       |—— electron-builder.yml # Config build .exe / .dmg
|       |—— release.yml
|
|—— README.md

```

Stratégie d'intégration des emails

Objectif

Définir un mécanisme unique, propre, extensible, pour connecter des boîtes mail, récupérer les emails, les normaliser, les analyser, et effectuer des actions (répondre, archiver, classer).

1. Aucune donnée envoyée à l'extérieur (IA locale + tokens stockés localement).
2. Un seul port EmailProvider, que tous les providers implémentent.
3. Normalisation systématique : tout email est converti vers EmailMessage.
4. Extensibilité : ajouter un fournisseur = ajouter un nouvel adapter.
5. Résilience : token Gmail automatiquement rafraîchi, gestion des erreurs, retry, logs.

Schéma

```

Frontend (Electron)
|
| REST local
v
Backend FastAPI
|
| → NadIA Core (MailboxService)
v
Ports (EmailProvider) ← abstraction commune pour tous les providers
|
+--> GmailAdapter
|
+--> IMAPAdapter   (futur)
|
+--> OutlookAdapter (futur)

```


Fonctionnement

- API Gmail REST
- OAuth2 avec refresh token

Workflow

- Au premier lancement , Nadia ouvre une fenêtre de selection de son fournisseur
- Seul choix possible pour le moment gmail
- Ouvre une seconde fenêtre d'autorisation de l'accès a gmail
- Utilisation du QuickStart google pour générer les credentials.json
- Stocker localement le token , token_gmail.json
- Chiffrer éventuellement le json

Gestion de la synchro

- Nadia ne va pas tout retélécharger a chaque fois . Voir pour du stockage local, via sqllite ou autre.
- Si des appels 401 sont retournés, rafraichir le token
- Détecter les nouveaux emails

Liste des fonctionnalités attendues

MVP (V1)

Gmail, résumé, importance, réponse, archiver, UI desktop

1. Connexion / Intégration email

- Connexion à une boîte Gmail via OAuth2
- Stockage local sécurisé du token (pas d'envoi externe)
- Synchronisation des emails récents
- Lecture du contenu d'un email (HTML → texte propre)
- Reconstruction des threads de conversation
- Mode lecture seule + mode lecture/écriture

2. Analyse automatique des emails

- Résumé automatique de chaque email via LLaMA 3 8B local
- Détection du sujet clé
- Extraction des éléments importants
- Identification du ton (optionnel)
- Génération d'un "quick digest" de la boîte mail

→ exemple : "12 emails aujourd'hui / 3 prioritaires / 2 nécessitent une action"

3. Classification & Priorisation

- Détection des emails importants (ImportanceService)
 - Classification automatique :
 - Important
 - À répondre
 - À archiver
 - Newsletter
 - Notification
 - Mise à jour en temps réel lors de la réception de nouveaux emails
-

4. Proposition de réponses

- Suggestion de réponses adaptées au contexte
 - Génération d'une réponse courte
 - Génération d'une réponse détaillée
 - Possibilité d'éditer la réponse générée avant envoi
 - Envoi via Gmail API
-

5. Actions sur les emails

- Archiver un email
 - Marquer comme lu / non lu
 - Répondre à un email
 - Ajouter des labels (si Gmail)
 - Supprimer localement mais pas sur Gmail (mode "ignorer")
-

6. Interface Desktop (Electron)

- Affichage Inbox
- Vue détail email
- Affichage résumé IA
- Affichage suggestion de réponse
- Boutons d'action (répondre, archiver, marquer lu)
- Indicateurs visuels d'importance
- Navigation fluide entre emails
- Mode "refresh inbox"

V2

IMAP, Outlook, recherche intelligente, recap quotidien, IA personnalisée

V3+

Extension web, tâches, analytics, organisation automatique